

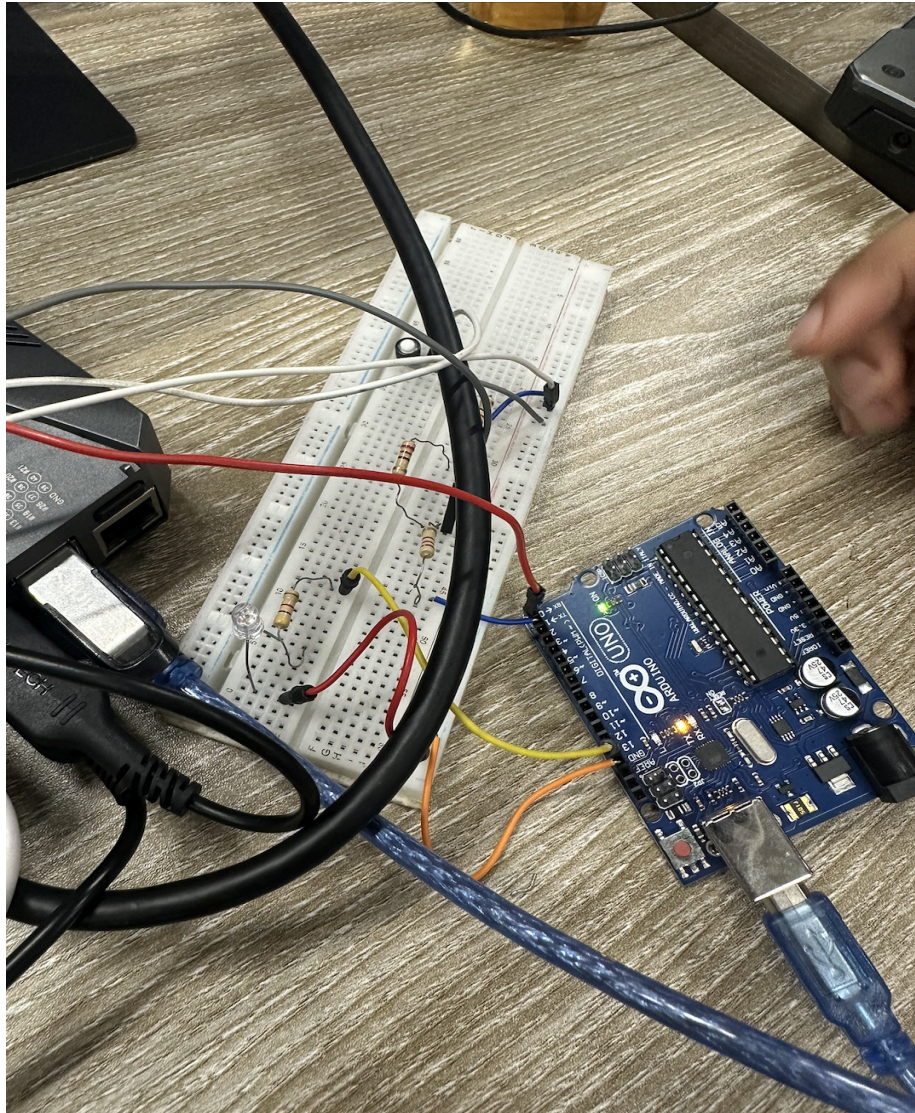
## **1. Objectives:**

- Understand and implement UART (Universal Asynchronous Receiver-Transmitter) communication between Arduino Uno and Raspberry PI and experience setting up the Arduino IDE on a Raspberry PI and enabling UART hardware communication.
- Interface the Arduino and Raspberry PI to work collaboratively as a hybrid embedded system.
- Using a voltage divider, explore voltage compatibility solutions between 3.3V (Raspberry PI) and 5V (Arduino) systems.
- Construct and design a circuit where an LED connected to the Arduino is controlled by a push button that is connected to the Raspberry PI.
- Deploy and develop cross-platform code to facilitate serial communication for data exchange and real-time I/O control.

## **2. Equipments:**

- Arduino Uno R3
- Raspberry PI 4
- LED (Light Emitting Diode)
- Push-button switch
- Resistor (220Ω for LED)
- Breadboard
- Jumper wires
- USB cable (for connecting Arduino to Raspberry PI)

### 3. Experimental Setup:



We used a push-button in the experiment and linked the Raspberry PI with the LED and connected the Raspberry PI and the LED with the Arduino Uno, and used UART as the communication protocol between the two. A push-button was attached to the GPIO pin 17 port of Raspberry PI as an input digital port. The LED was pointed to pin 13 of the Arduino using 220 resistor. In UART communications, Raspberry PI connector connections were the RX pin of the Raspberry PI to pin 0 of the Arduino (GPIO14) and the TX pin of the Arduino to pin 1 of the Raspberry PI (GPIO15) using a voltage divider to reduce the Arduino signal which was 5V to 3.3V. The two machines had a common terrain where communication could be stable. Both the Raspberry PI and the Arduino were given some code in order to identify button presses and send them back to make the LED blink every time a button was pressed. With this arrangement, real-time serial communications and control were established on two microcontroller platforms.

#### 4. Code: (If Applicable)

```
#Arduino Code:
const int ledPin = 13;
void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  Serial.println("Arduino is ready. Waiting for command...");
}

void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();

    if (command == '1') {
      Serial.println("Button pressed: Blinking LED...");
      digitalWrite(ledPin, HIGH); // Turn the LED on
      delay(1000); // Wait for 1 second
      digitalWrite(ledPin, LOW); // Turn the LED off
      Serial.println("LED blinked.");
    }
  }
}

#Raspberry PI Code:

import RPi.GPIO as GPIO
import serial
import time

# Set up GPIO for button
button_pin = 17 # GPIO pin 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(button_pin, GPIO.IN)

# Set up UART (Serial) communication with Arduino
ser = serial.Serial('/dev/serial0', 9600)

print("Raspberry Pi is ready. Press the button to blink the LED...")

try:
    while True:
        if GPIO.input(button_pin) == GPIO.LOW: # If button is pressed
            print("Button pressed! Sending signal to Arduino...")
            ser.write(b'1') # Send signal to Arduino to blink LED
            time.sleep(0.2) # Debounce delay
        else:
            print("Button not pressed. Waiting for press...")
            time.sleep(0.1) # Short delay to avoid excessive CPU usage
except KeyboardInterrupt:
    print("Program interrupted.")
finally:
    GPIO.cleanup()
    print("GPIO cleaned up. Exiting program.")
```

## **5. Results (Output of the experiment):**

This experiment allowed us to achieve UART communication between the Raspberry PI and Arduino Uno to allow real-time control of an LED via a push-button. The push-button pointed towards the Raspberry PI was switched on, and this triggered the Raspberry PI to recognize the input and pass the signal ('1') to the Arduino through the output pin (TX) to the input pin (RX). When this command came, the Arduino board interpreted the signal and reciprocated that by making the LED on digital pin 13 of that unit to blink. This affirmed that the UART messages were working well, and the two devices were aligned in the process. When the button was not switched, the LED was not lit, and when the button was switched on the LED blinked one second, which indicates the reliability of data transmission and response between two platforms. In general, the configuration confirmed our knowledge of voltage level shifting, GPIO setting, and serial data processing of a cross-platforms embedded system.

## **6. Discussions/Answers:**

Through the experiment we have been able to know how to set up proper UART communication between Arduino and Raspberry PI and also the need to have correct level voltage shifting as well as having the right settings in each of the pins. Among the challenges that we encountered we had on the Arduino board the TX/RX pins also handling the code upload and hence upload failures were experienced when connected to the Raspberry PI. We solved it by loading the code into it first and then connecting the UART pins. The other problem was whether the voltage in Arduino (5V) would lead to the damaging of Raspberry PI (3.3V); this was overcome by the use of voltage divider. In general, the lab helped us get knowledge about hybrid embedded systems and real-time serial communication.